

КУРС «ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ» · 1 СЕМЕСТР

04

Лекция 4

Функции: перегрузка, параметры, inline, RVO

Объявление и определение · передача параметров · перегрузка и её разрешение · default-аргументы · inline/constexpr · RVO и NRVO · указатели на функции · рекурсия · ADL

МАИ · лекции курса ООП

C++20 | Функции | 10 глав

Вспоминаем, прошлую лекцию

```cpp
#include <iostream>

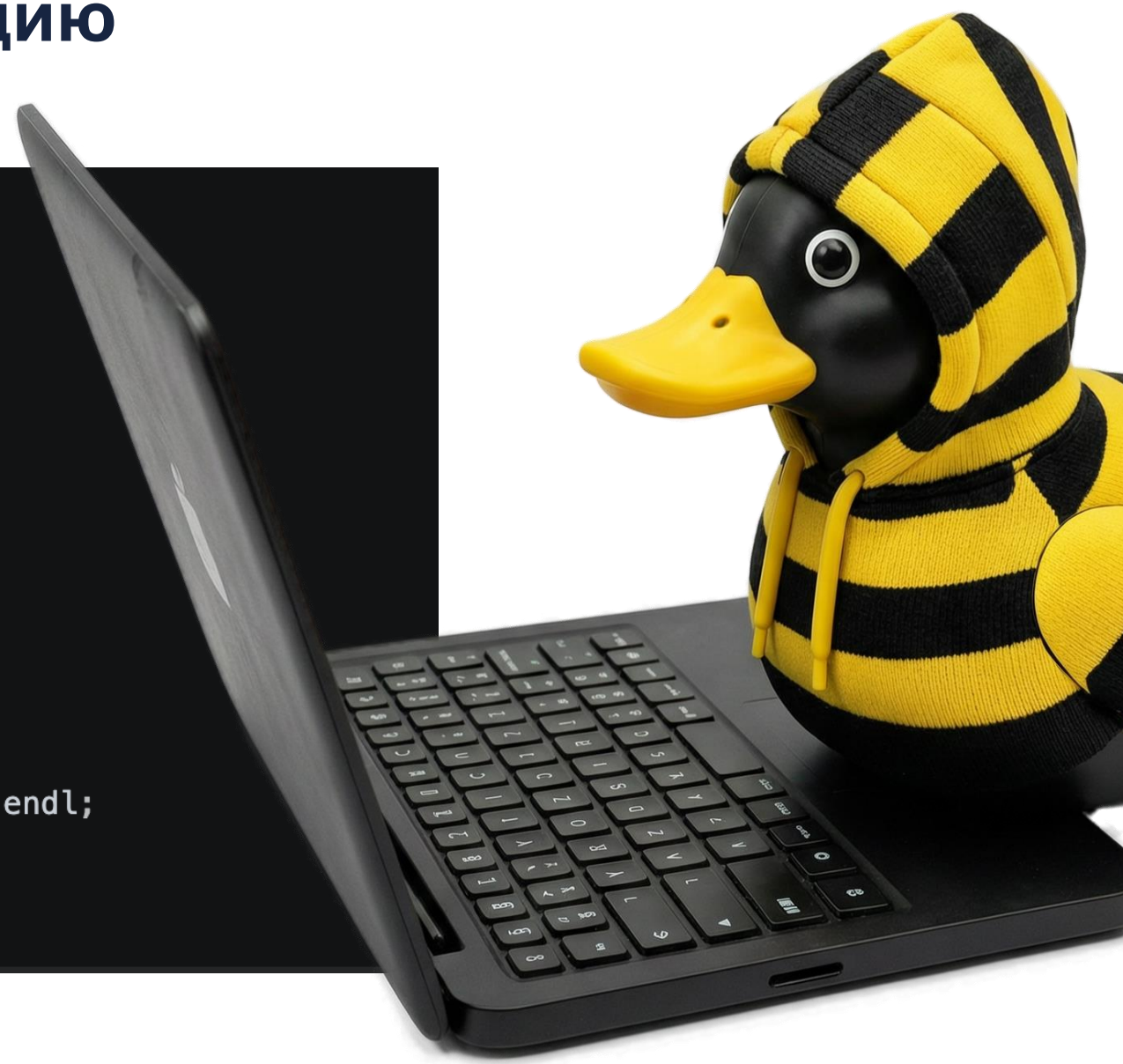
int value(const int & val) {
    return val ==0 ? 0 : value(val - 1);
}

int value(const int && val) {
    return val ==0 ? 0 : value(val);
}

// Зациклится ли программа?

int main() {
    std::cout << "Value: " << value(7) << std::endl;
    return 0;
}

```



ПЛАН ЛЕКЦИИ

Десять глав сегодняшней лекции

01 Объявление vs определение

02 Передача параметров

03 Перегрузка функций

04 Разрешение перегрузки

05 Аргументы по умолчанию

> **Сквозная идея:** функция — это и контракт (сигнатура), и поведение (тело), и значение (указатель). Всё это — про одну конструкцию языка.

06 inline и constexpr

07 RVO и NRVO

08 Указатели на функции

09 Рекурсия

10 ADL · Koenig lookup

Прототип сообщает, тело — делает

```
// math_utils.h — объявления (прототипы)
int add(int a, int b);
double calculate_area(double radius);

// math_utils.cpp — определения
int add(int a, int b) {
        return a + b;                    // тело
}

// main.cpp — компилятор видит объявление,
// линковщик найдёт определение
#include "math_utils.h"
int main() {
        return add(2, 3);
}
```

- **Объявление** — имя, типы параметров, тип возврата. Без тела.
- **Определение** — то же самое + тело.
- Объявлений может быть **сколько угодно** — они должны совпадать.
- Определений — **ровно одно** на программу (ODR).
- Исключение — inlineи шаблоны: могут жить в заголовке.

Разделяйте код

.h — интерфейс для клиентов, .cpp — реализация. Так заголовок можно включать куда угодно.

ПРИМЕР

Смотрим пример: 01_declaration_definition

```cpp
int square(int x);              // объявление (прототип)

int main() {
    std::cout << square(6);     // OK: вызов проверяется по объявлению
}

int square(int x) {             // единственное определение
    return x * x;
}
```



Четыре способа — четыре смысла

```
void inc(int x)              { x += 1; }                  // копия
void inc(int& x) { x += 1; }                              // оригинал
void inc(int* x) { if (x) *x += 1; } // nullptr-able

int a = 10;
inc(a);          // 10 — значение не изменилось
inc(a);          // 11 — изменился оригинал
inc(nullptr);              // безопасно

// Чтение без копирования — const ссылка:
void print(const std::string& s) {
    // s += "!";           // Ошибка: const!
}
print("World");            // rvalue привязывается к const&
```

Параметр	Когда
T	дешёвый тип или нужна копия
const T&	чтение без копирования (строки, объекты)
T&	функция модифицирует аргумент
T*	необязательный параметр

По умолчанию

const T&— основной способ передать «тяжёлый» объект: ни копии, ни модификации.

ПРИМЕР

Смотрим пример: 02_parameter_passing

```cpp
void process(std::string data) {   // копируется вся строка при каждом вызове
    std::cout << data;
}
```

```cpp
void process(const std::string& data) {   // только ссылка, копии нет
    std::cout << data;
}
```



Одно имя — разные списки параметров

```
int max(int a, int b) { return a                    > b ? a : b; }
int max(int a, int b, int c) {                              // кол-во
        return max(max(a, b), c);
}
double max(double a, double b) {                            // тип
        return a > b ? a : b;
}

void process(std::string& s) { ... }                        // non-const
void process(const std::string& s) { ... } // const

max(3, 7);                          // int,int
max(3, 7, 5);                       // int,int,int
max(3.14, 2.71);                    // double,double
process(s);                         // non-const версия
process(std::string()); // rvalue → const версия
```

- различают по **количеству**, **типу** и **квалификатору** параметров
- **тип возврата не участвует** в разрешении
- перегрузка по const— классика для методов

Ошибка

```
int    foo(int);
double foo(int);          // параметры совпали —
                          // не перегрузка, а ошибка
```

ПРИМЕР

Смотрим пример: 03_overloading

```cpp
int    parse_int(const std::string& s);
double parse_double(const std::string& s);
// или перегрузка по параметрам:
int    to_number(const std::string& s, int    tag);
double to_number(const std::string& s, double tag);
```



Компилятор выбирает лучшую в три этапа

```
void foo(int)    { /* ... */ } void foo(double) {
/* ... */ }void foo(long)    { /* ... */ }


foo(42);            // exact → foo(int)
foo(3.14);          // exact → foo(double)
foo(42L);           // exact → foo(long)
foo(3.14f);         // promotion float→double → foo(double)
// foo(42u); // unsigned → int u unsigned → long
                    // одинаково хороши → AMBIGUOUS!
```

- **1. Кандидаты** — все функции с именем в области видимости
- **2. Фильтрация** — отбрасываем недостижимые
- **3. Ранжирование** — выбираем по приоритету преобразований

Приоритет преобразований

exact match → **promotion** (short→int, float→double) → **standard** (int→double) → **user-defined** (конструкторы, операторы).

Несколько одинаково хороших — **ошибка компиляции** (ambiguity).

ПРИМЕР

Смотрим пример: 04_overload_resolution

```
// Приоритет преобразований (от лучшего к худшему):
//   1) точное совпадение
//   2) promotion (short->int, float->double, bool->int)
//   3) стандартное преобразование (int<->double и т.п.)
//   4) пользовательское преобразование
```



Не хочется передавать — не передавайте

```
void log(const std::string& message,
         const std::string& level = "INFO",
         bool timestamp = true)
{
    if (timestamp) std::cout << "[TIME] "; std::cout << "[" <<
    level << "] "
                   << message << "\n";
}

log("Server started"); log("Connection lost",          // INFO, true
"WARNING"); log("Debug data", "DEBUG", false);        // true
                                                      // всё явно
```

- умолчания — только **в конце** списка, справа налево
- указываются **один раз** — обычно в объявлении
- не сочетайте с перегрузкой — иначе неоднозначность

Ловушка

```
void foo(int a, int b = 10);
void foo(int a);                    // foo(5) — ambiguous!

int add(int a, int b);                          // объявление:
// int add(int a, int b = 0); // умолчание тут — 1 раз
```

ПРИМЕР

Смотрим пример: 05_default_arguments

```cpp
// Аргументы по умолчанию задаются СПРАВА НАЛЕВО и только в конце списка.
// Указываются один раз (здесь — в определении; обычно — в объявлении в .h).
void log(const std::string& message,
         const std::string& level = "INFO",
         bool timestamp = true) {
    if (timestamp) {
        std::cout << "[12:00] ";
    }
    std::cout << "[" << level << "] " << message << '\n';
}
```



Сборка на этапе компиляции

```cpp
// inline: определение можно держать в заголовке
inline int square(int x) { return x * x; }

// constexpr: вычисляется на этапе компиляции,
// если аргументы — константные выражения
constexpr int factorial(int n) {
    return n <= 1 ? 1 : n * factorial(n - 1);
}
constexpr int f5 = factorial(5); static_assert(f5
== 120);

// C++14: циклы и переменные разрешены
constexpr int fibonacci(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1;
    for (int i = 2; i <= n; ++i) {
        int t = a + b; a = b; b = t;
    }
    return b;
}
static_assert(fibonacci(10) == 55);
```

- inlineсегодня — это **правило линковки**, а не просьба инлайнить
- компилятор сам решает, встраивать ли вызов
- constexpr-функция вызывается и в рантайме, если аргумент не константа
- без inlineопределение в заголовке → **multiple definition**

С C++14 constexpr-функции могут содержать циклы, локальные переменные и условия — это уже «настоящий код» для этапа компиляции.

ПРИМЕР

Смотрим пример: 06_inline_constexpr

```cpp
constexpr int square(int x) { return x * x; }
int y = square(++a);   // ++a вычисляется один раз, проверка типов есть
static_assert(square(5) == 25);   // ещё и compile-time
```



Возврат по значению — без копирований

```
std::string create() {
    return std::string("hello");                    // RVO (prvalue)
}   // C++17 — гарантирована стандартом

std::string create_named() { std::string r =
    "hello"; r += " world";
    return r;
}                                                   // NRVO (обычно сработает)


std::string bad_nrvo(bool flag) {std::string a = "hello", b =
    "world";
    if (flag) return a;                 // два разных пути —
    return b;                           // NRVO не сработает
}

std::vector<int> bad_return() {
    std::vector<int> v{1,2,3};
    return std::move(v);                // ПЛОХО: запрещает NRVO!
}
```

- **RVO**: prvalue создаётся сразу в памяти вызывающей стороны
- с C++17 RVO для prvalue **обязательна**
- **NRVO**: то же для именованной переменной — не гарантирована
- NRVO: локальная переменная, тип совпадает, один объект на всех путях
- return std::move(local)— антипаттерн

Правило: возвращайте по значению return result;— компилятор применит NRVO или перемещение. Копия не появится.

ПРИМЕР

Смотрим пример: 07_rvo_nrvo

```cpp
std::cout << "make_prvalue():\n";
Tracker a = make_prvalue();

std::cout << "make_named():\n";
Tracker b = make_named();

std::cout << "make_with_move():\n";
Tracker c = make_with_move();
```



Поведение как значение

```
int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

using BinaryOp = int(*)(int, int);                    // тип

int apply(BinaryOp op, int x, int y) {
        return op(x, y);                               // вызов
}

BinaryOp op = add; op(3,          // имя функции → указатель
4);                               // 7
op = mul;          op(3, 4);      // 12

BinaryOp ops[] = {add, mul};                  // массив поведения
for (auto f : ops) f(5, 3);                   // "8 15"

// std::function — хранит любые вызываемые:
std::function<int(int,int)> f = add;
                                                       // лямбда
f = [](int a,int b){ return a - b; };                  // с захватом
int factor = 10;

std::function<int(int)> g = [factor](int x){
        return x * factor; };
```

Способ	+	–
void(*)(int)	нулевой overhead	нет захвата лямбд
std::function	универсален	heap, виртуальный вызов
шаблон	низкий overhead	шаблонный код

ПРИМЕР

Смотрим пример: 08_function_pointers

```cpp
int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

// Тип: указатель на функцию (int, int) -> int.
using BinaryOp = int (*)(int, int);

int apply(BinaryOp op, int x, int y) {
    return op(x, y);
}
```



Прямая, хвостовая и взаимная

```
// Прямая: базовый случай + рекурсивный
int factorial(int n) {
        if (n <= 1) return 1;                          // база!
        return n * factorial(n - 1);
}

// Хвостовая: вызов — последняя операция
int factorial_tail(int n, int acc = 1) {
        if (n <= 1) return acc;
        return factorial_tail(n - 1, n * acc);
}
// компилятор (-O2) превращает в цикл

// Взаимная: нужны предварительные объявления
bool is_even(unsigned n);
bool is_odd(unsigned n);
bool is_even(unsigned n) { return n==0 || is_odd(n-1); }
bool is_odd(unsigned n)                { return n!=0 && is_even(n-1); }
```

- **базовый случай** обязателен — иначе stack overflow
- хвостовая рекурсия → цикл при оптимизации
- взаимная требует **объявлений** до определений
- наивный fibonacci— экспоненциальная сложность

Внимание

для больших значений используйте итеративные алгоритмы — глубокая рекурсия переполняет стек.

ПРИМЕР

Смотрим пример: 09_recursion

```cpp
bool is_even(unsigned n) {
    if (n == 1) return false;
    return is_odd(n - 1);
}

bool is_odd(unsigned n) {
    if (n == 1) return true;
    return is_even(n - 1);
}
```



Функции ищутся рядом с аргументами

```
namespace geometry {
        struct Point { double x, y; };
        void print(const Point& p) {std::cout << "(" << p.x
                << ", "
                                << p.y << ")";
        }
        std::ostream& operator<<(std::ostream& os,
                                                const Point& p) {
                return os << "(" << p.x << ", " << p.y << ")";
        }
}

geometry::Point p{3.0, 4.0};


print(p); std::cout <<                  // ADL нашёл geometry::print
p;                                      // ADL нашёл operator<<
```

- поиск идёт в **пространствах имён типов аргументов**
- без ADL пришлось бы писать geometry::operator<<

Где критичен

Операторы (std::cout << x) · идиома swap· begin/endдля range-based for.

```
template <typename T>
void generic_sort(T& a, T& b) {
        using std::swap;                // видимый fallback
        if (a > b) swap(a, b);          // ADL → my::swap
}
```

ПРИМЕР

Смотрим пример: 10_adl

```cpp
int main() {
    // ADL находит geometry::describe, хотя мы не написали geometry::
    std::cout << "describe(Circle)       -> " << describe(geometry::Circle) << '\n';
    std::cout << "sides(Square)          -> " << sides(geometry::Square) << '\n';

    // Имя describe одно и то же, но ADL выбирает функцию по типу аргумента.
    std::cout << "describe(Meter)        -> " << describe(physics::Meter) << '\n';


    return 0;
}
```



ИТОГИ

Ключевые концепции лекции

- **Объявление** сообщает о функции, **определение** содержит тело. ODR — одно определение.
- const T&— основной способ передачи «тяжёлых» объектов.
- Перегрузка различает по параметрам, но **не по типу возврата**.
- **RVO/NRVO** убирают копирование при возврате; с C++17 RVO гарантирована.
- constexpr— вычисление на этапе компиляции при константных аргументах.
- **Указатели на функции** передают поведение как данные.
- **ADL** ищет функции в namespace аргументов — основа операторов и swap.
- inlineсегодня — правило линковки, а не просьба встроить.

> **Связка с практикой:** перегрузка clamp, функции высшего порядка transform_arrayи constexpr-вычисления — всё это пригодится в лабораторных работах.

ФИНАЛ

Вопросы для самопроверки

- Разница между объявлением и определением?
- Когда по значению, а когда по const&?
- Участвует ли тип возврата в разрешении перегрузки?
- Порядок предпочтения преобразований?
- Что такое RVO и NRVO? Когда NRVO не сработает?
- Почему return std::move(local)хуже return local?
- Чем std::functionотличается от указателя? Почему
- хвостовая рекурсия эффективнее обычной? Как ADL
- помогает в std::cout << x?
- Можно ли constexpr-функцию с циклом?
-